SLA-Based Profit Optimization Resource Scheduling for Big Data Analytics-as-a-Service Platforms in Cloud Computing Environments

Yali Zhao, Rodrigo N. Calheiros, Graeme Gange, James Bailey, and Richard O. Sinnott

Abstract—The value that can be extracted from big data greatly motivates users to explore data analytics technologies for better decision making and problem solving in various application domains. Analytical solutions can be expensive due to the demand for large-scale and high-performance computing resources. To provision online big data Analytics-as-a-Service (AaaS) to users in various domains, a general purpose AaaS platform is required to deliver on-demand services at low cost and in an easy to use manner. Our research focuses on proposing efficient and automatic admission control and resource scheduling algorithms for AaaS platforms in cloud environments. In this paper, we propose scalable and automatic admission control and profit optimization resource scheduling algorithms, which effectively admit data analytics requests, dynamically provision resources, and maximize profit for AaaS providers, while satisfying QoS requirements of queries with Service Level Agreement (SLA) guarantees. Moreover, the proposed algorithms enable users to trade-off accuracy for faster response times and less resource costs for query processing on large datasets. We evaluate the algorithm performance by adopting a data splitting method to process smaller data samples as representatives of the original big datasets. We conduct extensive experiments to evaluate the proposed admission control and profit optimization scheduling algorithms. Experimental evaluation shows the algorithms perform significantly better compared to the state-of-the-art algorithms in enhancing profits, reducing resource costs, increasing query admission rates, and decreasing query response times.

Index Terms—Resource scheduling, service aevel agreement, analytics-as-a-service, data splitting, big data, cloud computing

1 Introduction

The value that can be extracted from big data greatly motivates users to explore data analytics techniques and solutions [1], [2]. Big data analytics is an emerging trend for decision-making and problem solving in many application domains, such as healthcare and medication. Cloud computing [3], [4] greatly eases and benefits big data analytics by elastically provisioning heterogeneous and on-demand resources and services at infrastructure, platform, and application levels, which are provisioned in a pay-per-use basis with Service Level Agreement (SLA) guarantees. Big Data Analytics Applications (BDAAs) are usually targeted to specific domains and typically used by large enterprises [5]. BDAAs require large-scale computing, which can incur significant financial costs based on the consumed resources. As such, BDAAs are usually used by large enterprises or major research projects and are not suitable for occasional or individual usage.

- *Y. Zhao, J. Bailey, R. O. Sinnott are with the University of Melbourne, Melbourne, VIC 3010, Australia. E-mail: yalizhao.alice@gmail.com, {baileyj, rsinnott}@unimelb.edu.au.*
- *G. Gange is with the Monash University, Melbourne, VIC 3800, Australia. E-mail: gkgange@gmail.com.*
- *R. N. Calheiros is with the Western Sydney University, Sydney, NSW, Australia. E-mail: R.Calheiros@westernsydney.edu.au.*

Manuscript received 26 Sept. 2017; revised 17 Aug. 2018; accepted 18 Dec. 2018. Date of publication 27 Dec. 2018; date of current version 3 Sept. 2021. (Corresponding author: Yali Zhao.)
Recommended for acceptance by K. S. Candan.
Digital Object Identifier no. 10.1109/TCC.2018.2889956

To facilitate big data analytics for various domains, a general purpose Analytics-as-a-Service (AaaS) platform is required to lead the market and deliver on-demand services to users at low cost and in an easy to use way [6]. Our proposed AaaS platform brokers BDAAs from third party application providers and commodity resources, such as Virtual Machines (VMs) and storage from cloud resource providers. The AaaS platform automatically manages cloud resources for BDAAs based on user demand and provisions AaaS as consumable services. We focus especially on BDAAs that typically process read-only queries and leave out data consistency and security issues. The AaaS delivery model greatly benefits users with limited expertise to set up and manage large-scale cloud computing environments and enjoy on-demand AaaS with guaranteed SLAs on a pay per use basis.

Resource scheduling is a core function for AaaS platforms that aim to maximize profits by optimizing resource and query scheduling, increase market share by admitting more queries, and improve user satisfaction by delivering on-demand AaaS with SLA guarantees. AaaS platforms schedule resources to execute queries with guaranteed SLAs on QoS requirements of budgets and deadlines, which allows users to control analytical costs and get timely results for efficient decision-making. However, resource scheduling for big data analytics on AaaS platforms must tackle several challenges:

- Data analytics for big data with large volumes (i.e., terabytes, petabytes) can take considerable time.

2168-7161 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

This requires efficient resource management and task scheduling techniques that are capable to return query results before user-specified deadlines for fast and efficient decision-making, especially for application domains requiring timely decisions.

- Big data analytics requires large-scale compute and storage resources that can incur considerable resource costs. It is a necessity for users to obtain query results under controllable costs, especially for users with limited budgets.
- Resource requirements of online analytical requests fluctuate over time, which raises the requirement for automatic resource provisioning to react to dynamic query demands, which can be stochastic.
- Guaranteeing SLAs for AaaS is important for user satisfaction. Violating SLAs not only incurs penalty costs but also decreases user satisfaction. Effective and efficient admission control and resource scheduling is required to minimize/avoid SLA violations.
- Query response time is a major performance indicator of service delivery and should be minimized for better user experience. Transferring big data for processing can take considerable time and incur high resource costs and hence should be minimized. Data locality-aware scheduling techniques should be utilized to move the computing resources to the data by creating cloud resources at datacenters where the datasets reside.
- Maximizing profit and minimizing query response times while guaranteeing SLAs of service delivery should be supported. This is a multi-objective optimization problem. The profit optimization problem requires efficient and effective modeling and formulation for optimal resource scheduling.
- Data analytics requests with tight budgets and deadlines cannot be admitted if they cause significant reduction of profit, decrease user satisfaction, and lead to loss of customers. Among existing data analytics techniques is data splitting that enables users to trade-off accuracy for faster response times and less resource cost by processing smaller data samples as representatives of the original datasets. Efficient adoption of such data processing techniques in resource scheduling is required to process big data with limited budgets and times.

To tackle the above challenges, our research focuses on proposing efficient and automatic admission control and resource scheduling solutions to support AaaS platforms, to not only maximize the profit for AaaS providers, but also to guarantee SLAs and minimize query time for AaaS users. The *major contributions* of our work are:

- We model and formulate a multi-objective profit optimization and query time minimization resource scheduling problem.
- We propose admission control and profit optimization scheduling algorithms that are able to admit queries whose QoS requirements can be satisfied to deliver services with SLAs guarantees; maximize the profit for AaaS platforms by optimizing resource utilization; and minimize query response times with time-efficient query execution solutions.
- The optimization algorithms utilize automatic scheduling mechanisms that allow the AaaS platform to provide online data analytical services that react to dynamic variation of resource requirements based on realtime queries.
- The optimization scheduling algorithms utilize scalable scheduling machanisms, which elastically lease under-provisioned cloud resources while releasing over-provisioned resources to optimize utilization and minimize cost.
- The scheduling algorithms adopt data-aware scheduling to ensure the best data processing locality to minimize the network transfer times and costs.
- The scheduling algorithms enable users to trade-off accuracy for reduced response times and less cost. Applying data splitting and sampling techniques provides significant performance improvement for resource scheduling that enables AaaS platforms to process data samples compared to processing the original data.
- The scheduling algorithms support flexible query dependencies. A query can vary from an independent task to a complex dataflow. Queries that rely on results of other tasks can only be executed after their dependent tasks complete. Such flexible support for task dependencies enables the AaaS platform to handle tasks of different complexities.

We conduct extensive experiments to evaluate the admission control and resource scheduling algorithms. The results show that the optimization scheduling algorithms perform significantly better with regards to saving cost, enhancing profit, and reducing query processing time in comparison to related state-of-the-art algorithms. Results also show that after applying data splitting-based scheduling mechanisms, there is a significant performance improvement including enhancing profits, reducing resource costs, increasing admission rates, and reducing query processing times over traditional resource scheduling mechanisms, i.e., without data splitting.

This paper is a significant extension of our previous work [8]. Compared to our earlier work, we refine and enhance the modeling and formulation of the multi-objective optimization problem for resource scheduling to accommodate new features of task dependency and data splitting based scheduling. Moreover, we refine the architecture and framework of the AaaS platform by adding data splitter to allow splitting of large datasets and processing smaller data samples for quicker response and less cost. We also redesign the data splitting-based admission control and profit optimization algorithms to allow data splitting and task dependency based resource scheduling. Furthermore, we redesign the experiments and conduct extensive experiments to evaluate the performance of the data splitting-based scheduling algorithm and non data splitting methods. We survey the state-of-the-art research works and compare our work to these works with regards to various performance parameters. In summary, this paper makes a significant contribution on the applicability, efficiency, and performance improvement of the data splitting-based admission control and profit

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

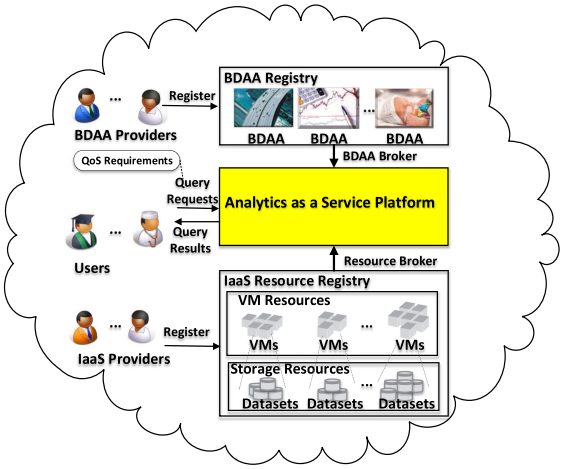

Fig. 1. Overview of the AaaS environment.

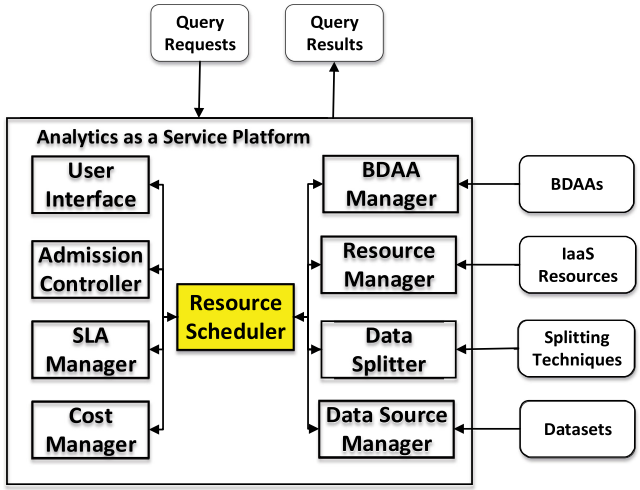

Fig. 2. Architecture of the AaaS platform.

optimization scheduling algorithms for AaaS platforms in cloud computing environments.

2 Problem Statement

2.1 Overview of the AaaS Environment

An overview description of the AaaS Environment is shown in Fig. 1.

Users submit queries online to the AaaS platform requesting particular data analytics services supported by specified BDAAs. These queries contain query specifications including QoS requirements on budgets and deadlines and data locations where to-be-analyzed datasets reside. The AaaS platform brokers cloud resources from third party Infrastructure-as-a-Service (IaaS) cloud providers while it also brokers BDAAs from third party application providers to provide analytical services based on application requirements of analysis requests. BDAAs are maintained in a BDAA registry, which is a catalog for BDAA providers to register for BDAAs to the AaaS market and also a directory for AaaS platforms to search for required BDAAs. At any time, a BDAA can be registered and deregistered from the BDAA registry. Thus, the AaaS platform needs to search for availability of the requested BDAAs before admitting queries. Similarly, cloud resources are registered and maintained in an IaaS resource registry. After the AaaS platform admits queries based on its admission policy, it makes profit optimization resource and query scheduling decisions to execute queries. Admitted queries are executed on allocated cloud resources by adopting query-scheduling solutions. Finally, the AaaS platform provisions AaaS to users by automatic and dynamic resource provisioning and query scheduling and deploys BDAAs on existing cloud resources for subsequent query processing.

2.2 Architecture of the AaaS Platform

The architecture for the AaaS platform is shown in Fig. 2, which includes the following components.

User interface serves as a mechanism for users and the AaaS platform to interact with each other. Users submit query specifications and negotiate SLAs with the AaaS platform. Details of the specifications are introduced in the modeling section. The AaaS platform outputs admission decisions and query results via the user interface.

Admission Controller is responsible for query admission. Queries that can be admitted must satisfy the following conditions: 1) the BDAAs requested by queries are available; 2) QoS requirements of queries can be satisfied based on current resource capacity and configuration; 3) to-be-processed datasets are accessible from data storage.

SLA Manager gets the queries after the admission control and builds SLAs for the admitted queries to ensure the AaaS platform can guarantee SLAs during service delivery to avoid SLA violations that incur additional penalty costs and cause user dissatisfaction.

Cost Manager manages all the costs in the AaaS platform including BDAA costs; cloud resource costs, AaaS service costs, and penalty costs. It provides pricing information to allow the Admission Controller to make performance estimation for query admission.

Resource Manager brokers IaaS resources from third party IaaS resource providers and keeps a catalog of all available resources from the different IaaS providers. The Resource Manager keeps monitoring the status of VMs, which are created, terminated, in-use, and idle. This information allows the Query Scheduler to take different actions accordingly, e.g., sending queries to VM instances that are idle for load balancing.

BDAA Manager is responsible for brokering BDAA resources from third party service providers based on user requests and maintaining application information. Information of BDAAs is maintained and updated in the BDAA Registry for application searching and selection.

Data Source Manager is responsible for managing to-be-processed datasets factoring in their size, type and location. We utilize cloud resources in datacenters where datasets reside to reduce data transfer times and costs.

Data Splitter is responsible for splitting large datasets to obtain smaller data samples for data analytics when budgets and deadlines are limiting factors. The data splitting can be achieved by different approaches such as data splitting and sampling. For queries that do not require full accuracy based on processing the entire datasets, the accuracy can be traded-off for faster response times and less resource costs by processing a less complete and hence less accurate subset

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

of the data. For queries that cannot be admitted due to tight deadlines or limited budgets, if users allow approximate query processing, the AaaS platform checks whether the query can be admitted by estimating the splitting time and cost together with the data processing time and cost compared to user-specified query deadlines and budgets. If applying data splitting techniques can process the data with SLA guarantees on QoS requirements, such queries can be admitted for processing.

Query scheduler is the central component of the AaaS platform and coordinates the other components. It makes query scheduling and resource provisioning decisions based on realtime resource and BDAA information the AaaS platform. It aims to make optimization scheduling decisions to maximize the profit, increase query admission rate, guarantee SLAs, while reacting to the dynamism of query workload and resource capacity. The query scheduler enables data locality-aware and dependency-based scheduling while supporting data splitting-based scheduling for faster response times and less resource costs.

2.3 Problem Modeling

The profit optimization resource scheduling problem aims to find the optimal resource scheduling solution to maximize profit for the AaaS platform and minimize query time, while delivering satisfactory services to users with SLA guarantees.

We consider a set of M queries $Qs = \{Q_1, Q_2, \ldots, Q_M\}$ submitted to the AaaS platform. A query can be a scan query, i.e., select the average body fat from a patient table, or a complex query, i.e., a machine learning algorithm to analyze the factors leading to lung cancer, etc. These queries request for a set of P Big Data Analytics Applications $BDAAs = \{BDAA_1,\ BDAA_2,\ \ldots,\ BDAA_P\}$ that execute on a set of N resources $Rs = \{R_1,\ R_2, \ldots, R_N\}$. A $BDAA$ can be a healthcare application running on top of data analytical framework Spark [9] to analyze patient health data. A resource can be a VM, storage, or a cluster of VMs, etc. Different cost policies are applied in the AaaS platform, such as policies for query costs and penalty costs.

Query $Q = \{SLA, CR, BDAA, CH, DE,\ AC\}$, where SLA denotes QoS requirements of budget (the maximum cost to execute a request) and deadline (the latest time to return the analytical result); CR denotes the requested cloud resources to execute a query task; $BDAA$ indicates the specific data analytics applications that users request to analyze the data; CH specifies the characteristics of the dataset, such as data size, data type, data distribution, and data locality; DE denotes the dependency requirement; AC denotes whether the accuracy of the query can be sacrificed. If users allow sacrificing accuracy during data analytics procedures when budget or deadline of a query is a limiting factor, the AaaS platform may enable processing representative datasets using data splitting techniques. To avoid expensive data transfer costs and long data transfer times, we move the computing resources to the data, hence we process data by creating VMs in the cloud datacenters where datasets are stored. We assume the data is prestored in datacenters and that storage and data transfer costs are paid directly by users.

Big Data Analytics Application $BDAA = \{AT, P\}$, where AT represents the application type while P represents the application profile, which contains mappings of the BDAA cost, required resource time including query time, sampling time, and splitting time, and resource configurations for different query requests. Obtaining profiles of heterogeneous BDAA applications requires expert knowledge from various application domains. Thus, we assume reliable BDAA profiles are provided by third-party BDAA providers. BDAAs provisioned by third party application providers should be accessible through a uniform interface to ease BDAA brokering and utilization.

Resource $R = \{RT,\ REC, NR, CC\}$, where RT denotes the resource type (e.g., memory optimized Amazon EC2 VM instances, such as an r3.large instance or a cluster of r3.large instances), REC represents the resource capacity (e.g., CPU, memory, storage, etc.) provided by the resource, NR represents the number of instances required to execute a query, while CC is the cost of each resource.

Cost $C = \{RC,\ BC,\ PC, ARC, PRO\}$, where RC denotes the overall resource cost of the AaaS platform, BC denotes the cost of BDAA applications that is assumed as a constant cost charged by BDAA providers, PC represents the penalty cost if any SLA of a given query is violated, ARC denotes the overall cost that users pay the AaaS platform for utilization of the data analytics services, and PRO denotes the overall profit made by the AaaS platform. We assume a constant service fee is charged for each query request depending on specific requested cloud resources, BDAA, and QoS requirements. Hence, ARC is a constant value.

2.4 Problem Formulation

The profit optimization scheduling problem is an NP-complete problem that can be polynomially reduced to an Integer Linear Programming (ILP) problem. We formulate the profit maximization resource scheduling problem based on the ILP model [10], [11]. Based on the cost model as introduced in the previous section, we can know ARC is a constant value. We aim to provide scheduling solutions with SLA guarantees on QoS requirements. Thus, PC is zero if SLAs are guaranteed. The PRO is calculated as $PRO = ARC - RC - AC - PC$. As ARC and AC are constant while PC is zero while SLAs are guaranteed, the objective of maximizing PRO is to minimize RC while minimizing the query processing time and delivering AaaS with SLA guarantees.

To achieve the scheduling objective, we have two individual objectives to fulfill: Objective A and Objective B. $A = minimize(\sum_{j=1}^{n} C_j * t_j)$, where j represents a cloud resource such as a VM or a cluster of VMs; n represents a set of resources; t_j is the number of hours to be used for a resource j; while C_j is the hourly cost of the resource j. Objective A aims to minimize the resource cost RC, calculated as the total costs of different resources, which is the product of purchased resource time t_j and the unit resource cost C_j. Objective $B = minimize(\sum_{i=1}^{m} s_i)$, where i represents a query request i; m denotes a set of query requests, while s_i is the start time of the query i. Objective B aims to find a query scheduling solution with the cost minimized resource configuration that can execute queries at the earliest time. This enables future queries to start execution at the earliest time s_i to reduce t_j and hence allows cost saving

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Definition of Symbols

Symbol	Definition
i	A specific query
j	A specific resource
m	The set of querie
n	The set of resources
t_j	The time to be purchased for resource j
T_j	The available resource time of resource j.
LRT_j	The remaining time of resource j in previous schedule
RRT_{ij}	The required resource time for query i on resource j
x_{ij}/x_{kj}	It indicates whether query i / query k is assigned to resource j
C_j	The unit cost of resource j
s_i/s_k	The start time of query i / query k
b_{ik}/b_{ki}	It indicates whether query i executes before query k
D_i	The deadline of query i
B_i	The budget of query i
C_{ij}	The cost of executing query i on resource j
EAT_j	The Earliest Available Time that resource j can be used to execute queries
TFT_j	The query finish time on created resource j
CT_j	The clock time of created resource j
RCT_j	The creation time of resource j

and minimizes the query times for quicker responses to improve the performance of delivered services. The symbols of the formulation are defined in Table 1.

We formulate the multiple Objective Z as a combination of Objective A and Objective B, which aims at minimizing resource costs while choosing a scheduling solution that starts query execution at the earliest time, subject to constraints (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12), (13), (14). The combination leads to a standard lexicographic optimization problem [12]. The importance of individual objectives contributing to the profit optimization resource scheduling problem is $A > B$. Coefficient F_0 is assigned to Objective A to guarantee the aggregated problem with minimization of all individual objective functions is consistent to the original profit optimization problem that any change of A dominate the changes of B [11], [12].

Objective Z:

$$Z = \text{minimize}\Big(F_0 * \sum\nolimits_{j\in n}(C_j * t_j) + \sum\nolimits_{i\in m} s_i\Big)$$

Subject to:

$$\sum_{i\in m,\ j\in n}(RRT_{ij} * x_{ij}) \le t_j + T_j \tag{1}$$

$$x_{ij} = \begin{cases} 1, & i\,scheduled\,on\,j \\ 0, & otherwise \end{cases},\ \forall\ i \in m;\ j \in n \tag{2}$$

$$b_{ik} + b_{ki} \le 1,\ \forall\ i, k \in m \tag{3}$$

$$b_{ik} = \begin{cases} 1, & i\,executes\,before\,k \\ 0, & otherwise \end{cases},\ \forall i,\ k \in m \tag{4}$$

$$b_{ik} + b_{ki} - x_{ij} - x_{kj} \ge -1,\ \forall\ i,\ k \in m; j \in n \tag{5}$$

$$s_i - s_k + F_1 * b_{ik} \le F_1 - RRT_{ij}, \forall\ i, k \in m;\ j \in n \tag{6}$$

$$s_i + \ F_2 * x_{ij} \le F_2 + D_i - RRT_{ij}, \forall i \in m,\ j \in n \tag{7}$$

$$s_i - t_j + F_3 * x_{ij} \le F_3 - RRT_{ij} + EPT_j,\ \forall\ i \in m,\ j \in n \tag{8}$$

$$s_i + x_{ij} * F_4 \ge F_4 + EAT_j,\ \forall\ i \in m,\ j \in n \tag{9}$$

$$C_{ij} * x_{ij} \le B_i,\ \forall i \in m,\ j \in n \tag{10}$$

$$\sum_{i\in m,\ j\in n} x_{ij} = 1 \tag{11}$$

$$x_{ij} = 0,\ \ if\ \ j\,cannot\,access\,dataset\,of\,i,\ \forall\ i \in m, j \in n \tag{12}$$

$$b_{ki} = 1,\ \ if\,i\,needs\,to\,process\,dataset\,split\,by\,k, \forall\,i, k \in m \tag{13}$$

$$b_{ki} = 1,\ \ if\,i\,is\,a\,dependent\,task\,of\,k, \forall\,i, k \in m \tag{14}$$

$$\left.\begin{matrix} x_{ij} = 1 \\ x_{kj} = 1 \end{matrix}\right\} \overset{either}{\Longrightarrow} \begin{cases} b_{ik} = 1, & b_{ki} = 0 \\ b_{ik} = 0, & b_{ki} = 1 \end{cases},\ \forall\ i, k \in m,\ j \in n \tag{15}$$

$$b_{ik} = 1 \overset{s_i}{\Rightarrow} +RRT_{ij} \le s_k,\ \forall\ i, k \in m,\ j \in n \tag{16}$$

$$F_1 \ge \max(s_i + RRT_{ij} - s_k) + 1, \forall\ i, k \in m,\ j \in n \tag{17}$$

$$s_i * x_{ij} + RRT_{ij} \le D_i, \forall\ i \in m, j \in n \tag{18}$$

$$F_2 \ge max(s_i + RRT_{ij} - D_i) + 1,\ \forall\ i \in m,\ j \in n \tag{19}$$

$$x_{ij} = 1 \Rightarrow s_i + RRT_{ij} \le t_j + EPT_j,\ \forall\ i \in m,\ j \in n \tag{20}$$

$$F_3 \ge \max(s_i - t_j + RRT_{ij} - EPT_j) + 1,\ \forall\ i \in m,\ \ j \in n \tag{21}$$

$$x_{ij} = 1 \Rightarrow s_i \ge EAT_j,\ \forall\ i \in m,\ j \in n \tag{22}$$

$$F_4 \le \min\big(s_i - EAT_j\big) - 1,\ \forall\ i \in m,\ j \in n \tag{23}$$

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

$$T_j = \begin{cases} LRT_j, \ j\,created \\ -RCT_j, \ otherwise \end{cases}, \ \forall j \in n \tag{24}$$

$$EAT_j = \begin{cases} \begin{cases} TFT_j, \ tasks\,running\,on\,j \\ CT_j, \ otherwise \end{cases}, \ j\ created \\ CT_j + RCT_j, \ otherwise \end{cases}, \ \forall j \in n. \tag{25}$$

Resource Capacity Constraints. Resource capacity constraints are used to ensure that the sum of required resource times of queries for resource j should be less than or equal to the available resource time of resource j, as shown in (1), where x_{ij} is a binary variable indicating whether query i is scheduled to resource j; RRT_{ij} is the Required Resource Time (RRT) to execute query i on resource j; t_j is the number of hours requested to purchase for resource j. When query i is scheduled to resource j, $x_{ij} = 1$; otherwise, $x_{ij} = 0$, as shown in (2). For a created resource, $\mathrm{T_j}$ is the Left Resource Time (LRT) before the end of purchased time in the previous schedule, denoted as LRT_j, which can be used to execute queries in the current schedule; while for newly created resources, T_j is a negative value that is used to deduct the Resource Creation Time (RCT) from t_j, denoted as $-RCT_j$, as shown in (24).

Query Execution Sequence Constraints. Query execution sequence constraints define the execution sequence to ensure the unique execution sequence of queries for the optimization solution, as shown in (3), where b_{ik} is a binary variable, which indicates whether query i is scheduled before query k (if query i is scheduled before query k, $b_{ik} = 1$; otherwise, $b_{ik} = 0$).

Query Deadline Constraints. Deadline constraints are used to ensure analytical solutions are returned before deadlines, as shown in Constraints (4), (5), (6), (7), (8), (9), (10). Constraint (4) is used to ensure the unique execution order of query i and query k by allowing only one of b_{ik} and b_{ki} to be 1. Constraint (5) ensures that query i must be executed either before query k or after query k when they are executed on the same resource j, which is derived from non-linear constraint (15) through linearization. Constraint (15) ensures that if i and k are scheduled on resource j, either $b_{ik} = 1,\ b_{ki} = 0$ (i executes before k) or $b_{ik} = 0,\ b_{ki} = 1$ (query i executes after query k). Constraint (6) is derived from non-linear constraint (16) using Big M method, which is used to ensure if $b_{ik} = 1$, query i should finish execution before k starts execution, where F_1 is a sufficient large constant satisfying (17) [6]. Constraint (7) is used to ensure that if query i starts execution on resource j at s_i, it should finish before its deadline D_i, derived from non-linear constraint (18), where F_2 is a sufficient large constant satisfying (19). Constraint (8) is derived from non-linear relationship (20), where F_3 is a sufficient large constant satisfying (21), which ensures that if query i is scheduled on resource j, its finish time has to be earlier than or equal to the end of the purchased time of resource j, which is the sum of previous purchased end time EPT_j of resource j and the new purchased time t_j. Constraint (9) is derived from non-linear relationship (22), where F_4 is a sufficient small constant satisfying (23), which is used to ensure that if i is assigned to resource j, it should start execution after EAT_j that is the Earliest Available Time (EAT) of resource j. If resource j is created while it has queries running on it, EAT_j is the Task Finish Time (TFT) of resource j, denot- ed as TFT_j; otherwise, EAT_j is the Clock Time (CT) of resource j, denoted as CT_j. If resource j is not created, EAT_j is the sum of CT_j and RCT_j, as shown in (25).

Algorithm 1. Admission Control Algorithm

```
Input: queries submitted by users
Output: admission or rejection decisions of queries
for each query ∈ submittedQueries
  requestedBDAA ← search BDAA registry
  if requested BDAA is available && dataset is accessible
    resConfigs ← search resource registry
    if resConfigs > 0
      for each configuration ∈ resConfigs
        EET = calculateEET
        EC = calculateEC
        if EET < deadline && EC < budget
          admQueries ← admit query
        end if
        else if query allows sacrifice accuracy
          apprQueries ← get approximate queries
          for each query in apprQueries
            DSC ← estimate resource cost for splitting
            DST ← estimate time for data splitting
            DPC ← estimate resource cost for splitting
            DPT ← estimate the splitting time
            if DSC+DPC < budget & DST+DPT < deadline
                admQueries ← admit the query
            end if
          end for
        end if
      end for
      if query is not admitted
        reject query with detailed rejection reasons
      end if
    end if
end for
```

Query Budget Constraints. Budget constraints are used to ensure that the execution cost of query i on j does not exceed the budget B_i, as shown in (10), where C_{ij} is the cost of executing i on j.

Query Scheduling Times Constraints. As shown in (11), the constraints on query scheduling times ensure that the sum of x_{ij} for a query must be 1 to ensure that the query i is scheduled to one resource for execution to avoid SLA violations.

Data Locality Constraints. Data locality constraints, as shown in (12) are used to ensure that queries can only be submitted to VMs where to-be-processed datasets are accessible to avoid expensive data transfer costs and long data transfer times. If a VM j does not have access to the dataset of a query i, $x_{ij} = 0$. This indicates that query i cannot be scheduled to VM j. For VMs that have access to the dataset of query i, x_{ij} will be allowed to be 1 only if such assignment can lead to a profit optimization scheduling solution.

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

Data Splitting Constraints. Data splitting on very large datasets is required when data analytics tasks are restricted with tight deadlines or limited budgets. Data splitting and sampling techniques aim to generate small data representatives of original large sdatasets. Processing smaller data samples can reduce resource costs while returning analytical results in a time-efficient manner. If large datasets are to be processed in such way, $b_{ki} = 1$, as shown in constraint (13), which indicates a data processing task i has to be executed after data splitting task k finishes the data splitting job.

Query Dependency Constraints. Query dependency constraints are used to define the task dependencies, as shown in (14), $b_{ik} = 1$ is set when query i depends on the execution results of query k. This is to ensure a query can only start execution when its dependent queries finish execution. Query dependency constraints allow defining simple task dependencies, such as bag of tasks, as well as complex task dependencies, such as dataflows.

3 Admission Control and Resource Scheduling

In this section, we introduce the admission control and resource scheduling algorithms, which admit queries and make resource scheduling decisions to allow the AaaS platform to achieve profit optimization objectives with minimized query processing times and SLA guarantees.

3.1 Admission Control

Admission control is the prerequisite for the AaaS platform to make effective and efficient query and resource scheduling decisions. The admission controller aims to maximize the query admission rate for higher market share while ensuring all the admitted queries can be successfully executed with SLA guarantees. It only admits queries whose QoS requirements can be satisfied with provisioned BDAAs and resources.

There are two possible scenarios where queries can be admitted. *Scheduling Scenario 1*: as we have considered in previous work [8], queries have sufficient budgets and deadlines, which allows queries to be executed by processing the entire datasets. *Scheduling Scenario 2*: users have limited budgets to spend and tight deadlines to obtain results. In this way, traditional data processing on the original datasets cannot fulfill QoS requirements for user queries. To face the challenge, data splitting techniques are utilized to obtain samples of the original datasets that enables quicker processing of queries on smaller datasets with reduced costs. After admission control, if queries cannot be admitted by the AaaS platform, the reasons for rejection are presented to users to give them the chance to modify their query specifications for resubmission.

The admission control algorithm works in the following way, as shown in Algorithm 1. It gets online queries submitted by users. Then, it iteratively makes admission decisions for each query. The admission control algorithm first checks whether the BDAA requested by the query is available and the dataset is accessible. If the conditions are satisfied, the admission controller further checks whether the QoS requirements of a query can be satisfied by any resource configuration by searching all possible Resource Configurations *(resConfigs)* in the resource registry. The Scenario 1 of the admission control is shown in Lines 3-11. For each resource configuration, it calculates the Estimated Execution Time (EET) and Estimated Cost (EC) to execute the query on such configuration at the earliest time without queuing the query. If such a configuration exists, the query is admitted.

If a query cannot be admitted due to limitations of budgets and/or deadlines, it is then further considered for approximate processing under the condition that the accuracy of the queries can be traded-off for quicker times and less costs, as shown in Lines 12-24. For each approximate query, admission control is conducted to estimate whether applying a data splitting method can satisfy its QoS requirements. The admission controller re-estimates the overall data processing cost as a result of the Data Splitting Cost (DSC) and the Data Processing Cost (DPC) while the overall data processing time as a result of the Data Splitting Time (DST) and the Data Processing Time (DPT). If a query can be executed by approximately processing split smaller datasets, it is admitted to the AaaS platform as shown in Scenario 2. After the splitting-based admission control, all the queries that cannot be admitted are given rejection reasons, as shown in Lines 25-27.

3.2 Resource Scheduling

In this section, we propose the profit optimization algorithm based on the ILP formulation of the resource scheduling problem. We compare the performance of the optimization algorithm against several heuristic algorithms. We also propose the data splitting-based profit optimization algorithm to enable big data processing under limited budgets and tight deadlines.

3.2.1 Profit Optimization (PO) Scheduling Algorithm

We build the PO resource scheduling algorithm, as shown in Algorithm 2, to maximize profit for the AaaS platform while processing queries with SLA guarantees and minimized processing times. PO first gets all submitted *queries* for each BDAA and makes query admission control decisions, as shown in Lines 1-3. Afterwards, it checks whether any resource configuration can process the admitted queries, *admQueries*, while satisfying QoS requirements. If such configurations are available, PO then selects the cost minimized configurations as the *resources* to execute queries. For *admQueries*, PO uses a heuristic method *selectResourceHeuristic* to decide the resource inputs, *heuResources*, for the *POScheduling* method (Lines 4-6). The *POScheduling* method adopts the ILP formulation of the resource scheduling problem with Objective Z subject to Constraints (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12). For extremely large query arrival rates, if no feasible solution is returned by *POScheduling* within a scheduling timeout, the heuristic algorithm is utilized to avoid deadline violations (Lines 8-14). After adopting the scheduling solution, PO scales resources down by periodic checks. During a check, if

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2. PO Resource Scheduling Algorithm

Input: submitted queries, resource configurations, existing resources, and BDAAs
Output: scheduling solution
1: **for** each BDAA $\in$ requested BDAAs
2: queries $\leftarrow$ get all submitted queries of BDAA
3: admQueries $\leftarrow$ admissionControl (queries, resConfigs, BDAA)
4: **if** (admQueries >0)
5: resources $\leftarrow$ get existing resources of BDAA
6: heuResources $\leftarrow$ **selectResourceHeuristic** (resources, resConfigs, admQueries, BDAA)
7: solution $\leftarrow$ **POScheduling** (admQueries, heuResources, BDAA)
8: **if** feasible solution is returned within scheduling timeout
9: adopt scheduling solution to execute queries
10: **end if**
11: **else**
12: solution $\leftarrow$ **maximumDelayTime** (heuResources, admQueries, BDAA)
13: adopt scheduling solution to execute queries
14: **end else**
15: resources $\leftarrow$ get existing resources of BDAA
16: scaleDown(resources)
17: **end if**
18: **end for**
19: **Procedure**: **selectResourceHeuristic** (resources, resConfigs, admQueries, BDAA)
20: avaResources $\leftarrow$ get available resources
21: exeResources $\leftarrow$ select resources from avaResources
22: **maximumDelayTime** (admQueries, exeResources, BDAA)
23: **if** any query not scheduled in admQueries
24: newResources $\leftarrow$ **newResourceHeuristic** (resConfigs, admQueries, BDAA)
25: heuResources $\leftarrow$ exeResources + newResources
26: **end if**
27: **Procedure**: **maximumDelayTime** (admQueries, heuResources, BDAA)
28: sort admQueries based on MDT
29: schedule admQueries to resources with minimal EST
30: **Procedure**: **newResourceHeuristic** (resConfigs, admQueries, BDAA)
31: modifications $\leftarrow$ get all possible modifications
32: newResources $\leftarrow$ searchHeuristic (resConfigs, modifications, admQueries, BDAA)
33: **Procedure: POScheduling** (admQueries, heuResources, BDAA)
34: getUpdatedInfo(admQueries, heuResources, BDAA)
35: defineScheduleObj(Objective Z)
36: defineScheduleConstraints (Constraints 1-12)
37: defineTimeOut(timeout)
38: solveScheduleProblem()
39: solution $\leftarrow$ getScheduleSolution()

Algorithm 3. SPO Resource Scheduling Algorithm

Input: submitted queries, resource configurations, existing resources, and BDAAs
Output: scheduling solution
1: **for** each BDAA $\in$ requested BDAAs
2: queries $\leftarrow$ get all submitted queries of BDAA
3: admQueries $\leftarrow$ admissionControl (queries, resourceConfigurations, BDAA)
4: apprQueries $\leftarrow$ get approximate queries
5: **for** each query in approximateQueries
6: dataSplittingTasks $\leftarrow$ generate data splitting tasks
7: **end for**
8: anaTasks $\leftarrow$ admQueries + dataSplittingTasks
9: **if** (anaTasks > 0)
10: resources $\leftarrow$ get existing resources of BDAA
11: heuResources $\leftarrow$ **selectResourceHeuristic** (resources, resConfigs, anaTasks, BDAA)
12: solution $\leftarrow$ **SPOScheduling** (anaTasks, heuResources, BDAA)
13: **if** feasible solution is returned within scheduling timeout
14: adopt scheduling solution to execute queries
15: **end if**
16: **else**
17: solution $\leftarrow$ maximumDelayTime (heuResources, anaTasks, BDAA)
18: adopt scheduling solution to execute queries
19: **end else**
20: resources $\leftarrow$ get existing resources of BDAA
21: scaleDown(resources)
22: **end if**
23: **end for**
24: **Procedure: SPOScheduling** (anaTasks, heuResources, BDAA)
25: getUpdatedInfo(anaTasks, heuResources, BDAA)
26: defineScheduleObj(Objective Z)
27: defineScheduleConstraints(Constraints 1-14)
28: defineTimeOut(timeout)
29: solveScheduleProblem()
30: solution $\leftarrow$ getScheduleSolution()

there is any resource that is idle and reaching the end of its billing period, the PO scheduling algorithm terminates such VMs to reduce resource costs (Lines 15-18).

The *selectResourceHeuristic* method (Lines 19-26) significantly reduces Algorithm Running Time (ART) of *POScheduling* by reducing the search space to allow it to efficiently generate scheduling solutions. It first selects the available resources, *avaResources*, from existing resources; then, it selects resources that can execute at least one query satisfying its QoS requirements from *avaResources*, as *exeResources*; afterwards, it utilizes *maximumDelayTime*-based scheduling method to maps *adQueries* to *exeResource*. The *selectResourceHeuristic* method combines *exeResources* and *newResources* as the *heuResources* to input to the *POScheduling* method. The *heuResources* provisions sufficient resource capacity to allow *POScheduling* to schedule queries with SLA guarantees. The capacity provided by *heuResources* is close to the optimal resource configuration that can be found by *POScheduling*, which greatly reduces the search space to find the optimal resource configuration and allows *POScheduling* to return timely profit optimization resource scheduling solutions.

The *maximumDelayTime* method (Lines 27-29) adopts a Maximum Delay Time (MDT) based scheduling method. MDT is the maximum time a query can be delayed for execution, which is the difference between query deadline and its execution time. The *maximumDelayTime* method first sorts queries in the MDT ascending order. Then, it keeps

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

mapping queries with minimal MDT first to resources that can satisfy their SLAs and give them the Earliest Starting Time (EST). After the MDT-based scheduling, if there is any query not scheduled due to limited resource capacity, the *newResourceHeuristic* method is utilized to create *newResources* to execute queries.

The *newResourceHeuristic* method (Lines 30-32) applies a search heuristic, *searchHeuristic*, to find the local cost-minimal resource configuration to create new resources for queries. The set of all possible resource configurations for a requested BDAA is represented by a directed acyclic graph, $Configuration = (Node, Edge)$, where $Node$ is a set of nodes while $Edge$ is a set of edges. A node is a possible resource $configuration_i$. An edge $(configuration_i, configuration_{i+1})$ indicates by applying a *modification*, $configuration_{i+1}$ can be obtained from $configuration_i$. The number of resource types available in the AaaS platform determines the number of possible $modification$ s, which include: adding the cheapest resource, adding more expensive resource, utill adding the most expensive resource. The *newResourceHeuristic* method gets all possible $modifications$ and iteratively applies each $modification$ on current resource configuration to generate a new configuration to execute queries. It continues these steps until it finds the cost-minimal local-optimal resource configuration, *newResources*, to execute queries with guaranteed SLAs.

The *ProfitOptimizationScheduling* method (Lines 33-39) first gets updated information of *heuResources*, *admQueries*, and BDAA. Afterwards, it creates the ILP solver with defined Objective Z and defined Constraints (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12). Afterwards, it specifies a timeout as the maximum time that the ILP solver can run. Then *POScheduling* utilizes the ILP solver to find the optimal solution for the scheduling problem. If a feasible solution is found, it is then applied in the AaaS platform to provision resources and schedule queries. Otherwise, the MDT-based scheduling method is utilized to schedule *admQueries* to *heuResources*.

3.2.2 Heuristic Greedy Search (HGS) Algorithm

The HGS scheduling algorithm is a combination of the *selectResourceHeuristic* method and the *maximumDelayTime* method. After admitting queries, the HGS scheduling algorithm first finds *heuResources* by applying the *selectResourceHeuristic* method and heuristically schedules queries to *heuResources* by applying the *maximumDelayTime* scheduling method. If the *POScheduling* method of the PO scheduling algorithm cannot return any feasible scheduling solution with defined timeout, the *maximumDelayTime* method is utilized as the scheduling method for PO to schedule queries in a heuristic way to avoid deadline violations. In this particular scenario, the scheduling solution generated by the PO scheduling algorithm is the same as the HGS scheduling algorithm.

3.2.3 Adaptive ILP (AILP) Algorithm

The AILP scheduling algorithm [6] is a two-phase resource scheduling algorithm, which aims to provide cost-effective resource scheduling solution to execute queries with SLA guarantees on budgets and deadlines. AILP provisions a scalable resource scheduling solution by applying a scheduling method based on its ILP modeling for two scheduling phases. In Phase 1, AILP aims to maximize resource utilization of existing resources and scale resources down by terminating existing resources when resources are overprovisioned to minimize resource cost with SLA guarantees. In Phase 2, if there is any query not successfully scheduled due to the under-provision of resources, AILP scales resources up to execute queries by creating resources with minimal resource cost while satisfying SLAs. To avoid unpredicted arrival of large number of queries, AILP specifies timeout to limit algorithm running time as latest time ILP-based scheduling method can run. If the timeout is exceeded when no feasible solution is returned, AILP utilizes *maximumDelayTime* scheduling method to make scheduling decision to avoid deadline violations. If the ILP-based scheduling method of AILP cannot return feasible solution, AILP generates the same solution as the solution generated by running AGS scheduling algorithm directly in this particular scenario.

3.2.4 Adaptive Greedy Search (AGS) Algorithm

The AGS scheduling algorithm [6] is a two-phase scheduling heuristic, which applies *maximumDelayTime* and *searchHeuristic* methods. After admission control, AGS schedules *admQueries* to existing resources by applying *maximum DelayTime* scheduling method in Phase 1. If created resources have limited capacity to execute queries satisfying QoS requirements, AGS then creates new VMs to process queries in Phase 2 by applying the *searchHeuristic* method to find local cost-minimal configuration of resources, *newResource*, and schedules queries to *newResource* based on the *new ResourceHeuristic* method by applying a local cost-minimal resource configuration.

3.2.5 Data Splitting-Based Profit Optimization (SPO) Scheduling Algorithm

For big data analytics with limited budgets or tight deadlines, such analytical requests cannot always be admitted due to QoS constraints. To tackle this challenge, data splitting-based scheduling algorithms can be applied to process a subset of the data and return analytical solutions with less costs and faster responses.

SPO applies SLA and data splitting based scheduling that allocate resources to execute queries on smaller split data samples with SLA guarantees. SPO enables processing queries with tight deadlines and limited budgets and allows the analytical results to be returned in a timely manner with controllable costs. It also benefits users who have cost-saving purposes and want timely results where accuracy is not a limited factor for the tasks. The presudo code of the SPO algorithm is shown in Algorithm 3.

The scheduling scenario for SPO works as follows: users submit data analytics requests to the AaaS platform requesting BDAAs to process their datasets. For each BDAA, the SPO scheduler firstly gets all the queries, resources running the BDAA, and the current resource configuration of the AaaS platform as input data. The resource scheduler then calls the admission controller to admit queries. After admission control, SPO schedule queries based on the SPO

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

Scheduling method, which adopts the ILP formulation of the resource scheduling problem with Objective Z subject to Constraints (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12), (13), (14).

If there are queries that cannot be admitted due to constraints of budgets and deadlines to process the large datasets, the Admission Controller checks among these queries and tries to re-admit the approximate queries, *apprQueries*, which allow approximate processing. If applying the splitting technique leads to satisfactory service delivery with respect to QoS requirements of queries, such queries are admitted. Afterwards, SPO scheduler generates the data analytics tasks, *anaTasks*, including data splitting tasks and data processing tasks, as shown in Lines 1-8. SPO then selects *heuResources* by applying *selectResourceHeuristic* method. Afterwards, the SPO scheduler applies *SPOScheduling* to generate profit-optimal scheduling solution, as shown in Lines 9-12.

The *SPOScheduling* method (Lines 24-30) first gets the up-to-date information regarding the BDAA, queries, and resources as input to the optimization solver. Then, SPO specifies the objective and constraints of the scheduling problem. SPO shares the same objective function as PO while accommodating additional constraints for data splitting and data dependencies. Finally, SPO utilizes the solver to generate the optimized scheduling solution. If feasible solutions are genereated within scheduling timeout, the solution is applied to execute queries. Otherwise, the heuristic scheduling approach *maximumDelayTime* is utilized to generate alternative solution to schedule resources (Lines 13-19). After applying the scheduling solution to execute queries, the SPO scheduler further conducts periodic check to scale resources down (Lines 20-21). After the current schedule, new periodic schedules are triggered as more online queries submitted by users arrive at the AaaS platform and need to be scheduled and processed with SLA guarantees.

4 Performance Evaluation

The admission control and profit optimization scheduling algorithms aim to maximize the profit for AaaS platforms and deliver AaaS services with SLA guarantees. In this section, we conduct extensive experiments to evaluate the effectiveness and efficiency of the proposed algorithms in admission control, SLA guarantee, profit enhancement, cost saving, resource configuration, algorithm running time, and query admission.

4.1 Experiment Setup

4.1.1 Experiment Environment

To evaluate the proposed admission control and resource scheduling algorithms, we build the AaaS framework in CloudSim [22], which is a discrete event simulation platform that enables modeling and simulation of cloud environments with on-demand resource provisions and configurations. It allows us to conduct repeatable and controllable evaluations of our proposed resource scheduling solutions without incurring expensive IaaS resource and BDAA costs. We utilize IBM ILOG CPlex 5.5 as the integer linear programming solver. CPlex is an optimization software package, which is an analytical decision support toolkit for rapid development and deployment of optimization models using mathematical and constraint programming [21]. We study algorithm performance for different scheduling scenarios including periodic scheduling that schedules queries for each Scheduling Interval (SI) and non-periodic (realtime) scheduling that schedules queries based on arrival times.

4.1.2 Resource Configuration

We simulate a datacenter that consists of 500 physical nodes. Each node has 50 CPU cores, 100 GB memory, 10 TB storage, and 10 GB/s bandwidth. We simulate five types of memory optimized VMs based on the Amazon EC2 VM model. The VMs are r3.large, r3.xlarge, r3.2xlarge, r3.4xlarge, and r3.8xlarge [13]. The resource configuration to execute queries can be a single VM or a cluster of VMs. A resource is charged based on the hourly utilization. The unit for memory is GiB, for storage is GB, and for the resource cost is dollar/hour.

4.1.3 Data Analytics Workload

The data analytics workload generated in this study is based on Big Data Benchmark [14] and a Normal Distribution (ND) based data splitting work [7]. The ND based data splitting technique [7] is utilized to split the original large datasets to obtain smaller split datasets to process. It provides significant performance improvement compared to process the original datasets. The ND-splitting work details the resource configurations and times to split large datasets using ND based data splitting applications while, the Big Data Benchmark details query processing times and the resource configurations using the Amazon EC2 Cloud to run queries on large datasets using different data analytic frameworks with given data size, data type, and data location. Based on the data splitting and processing information, we model the time for data processing and splitting on BDAAs under different resource configurations.

Each query request contains the following information: *Submission time* which is generated using a Poisson process with 1 minute mean Poisson arrival interval. *Query type* includes scan query, aggregation query, join query, and User Defined Function (UDF) query. *BDAAs*: 4 types of BDAAs are considered in our experiment that are built on Impala (disk) (BDAA1) [15], Shark (disk) (BDAA2) [16], Hive (BDAA3) [17], and Tez (BDAA4) [18]. *Resource time* is modeled based on the resource requirements of queries.

As the performance of queries can vary, we model 10 percent performance variation by introducing a variation coefficient [19], which is generated from a Uniform Distribution with upper bound 1.1 and lower bound 0.9. *User*: we simulate 50 users who submit queries and request for BDAAs. *Deadline* is generated as the benchmark does not contain QoS requirements on deadline, so is budget. Two types of deadline are considered, tight deadline and loose deadline. Tight deadline is generated using a Normal Distribution (3, 1.4). Loose deadline is generated similarly using a Normal Distribution (8, 3) [20]. Similarly, *Budget* contains tight budget and loose budget. Tight budget is generated using a Normal Distribution (3, 1.4) while loose budget is generated using a Normal Distribution (8, 3).

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Query Admission and SLA Guarantees

Rate	Real Time	Periodic					
		10	20	30	40	50	60
Admission	84.0	79.3	74.8	71.8	68.5	65.3	63. 0
SLA Guarantee	100	100	100	100	100	100	100

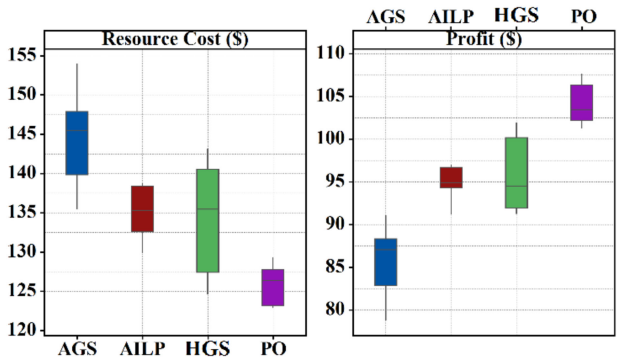

Fig. 3. Median profits and resource costs of PO, HGS, AILP, and AGS algorithms.

4.2 SLA Based Resource Scheduling

We generate an approximate 7 hours query workload on 30 GB data that contains 400 queries. We conduct the realtime scheduling and periodic scheduling by varying the SI from 10 to 60 (unit: minute) to evaluate the algorithm performance regarding admission control, SLA guarantee, profit enhancement, resource cost, resource configuration and algorithm runtime.

4.2.1 Admission Control and SLA Guarantees

We conduct the realtime scheduling and periodic scheduling by varying the SI from 10 to 60 (unit: minute) to evaluate the algorithm performance regarding admission control and SLA guarantee. We calculate the query admission rate and SLA guarantee rate based on the obtained results, as shown in Table 2. For realtime scheduling, the admission rate is 84.0 percent. For periodic scheduling with SI varied from 10 to 60, the admission rate decreases from 79.3 to 63.0 percent since the longer SI is the more queries are rejected due to deadline constraints. Thus, shorter SIs are recommended as more queries can be admitted to increase user satisfaction and enlarge market share. All admitted queries are successfully executed with 100 percent SLA guarantees, which is the result of effective and efficient admission control and resource scheduling. Service delivery with SLA guarantees helps to increase user satisfaction and reputation of AaaS providers.

4.2.2 Profit Enhancement and Cost Saving

We evaluate the mean and median resource costs and profits of PO compared to HGS, AILP, and AGS for all scheduling scenarios, as shown in Figure 3. Compared to HGS, AILP and AGS, PO saves 7 to 15 percent more resource costs respectively while creating 9 to 19 percent higher profit. Moreover, the mean profit created by PO is $104 and the resource cost is $126. Compared to HGS, AILP, and AGS, the profit of PO is [9, 10, 21 percent] higher while the resource cost is [7, 7, 15 percent] less. The above results indicate the overall performance advantages of PO in achieving higher cost saving and profit enhancement objectives.

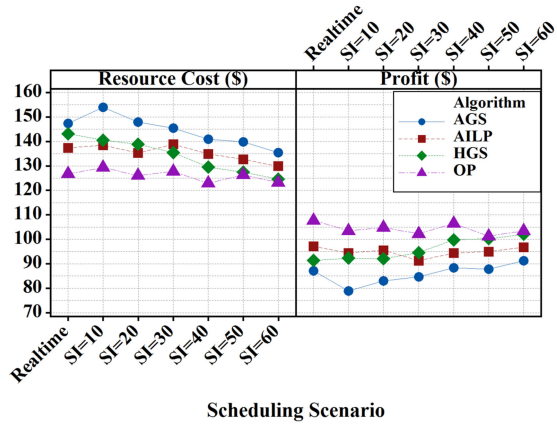

Fig. 4. Profits and resource costs of PO, HGS, AGS, and AILP algorithms.

1) Profits and Resource Costs for Different Scenarios. We detail the performance evaluation of PO in different scheduling scenarios, as shown in Fig. 4. For realtime and periodic scheduling, results indicate that PO creates the highest profit and generates the least cost while AGS performs the opposite. As an optimization algorithm, PO performs significantly better than AILP to generate profit optimization solutions. As a heuristic algorithm, HGS performs significantly better than AGS that serves as a suitable alternative algorithm to deliver cost-effective scheduling solutions for extreme large-scale arrivals of queries. From this section on, we do not include AGS and AILP in the evaluation for simplicity purposes.

The obtained results also show that PO creates higher profit and generates less resource cost compared to HGS for both realtime and periodic scheduling. As SI increases, due to high query arrival rates, the scheduling decisions generated by PO for some schedules may not be the optimal ones, which weaken the advantages of PO.

SI can be adjusted to a suitable scale based on the arrival rate of queries for different workloads to enable PO to generate timely profit optimization solutions. To achieve higher cost saving and profit enhancement, while admitting more queries, periodic scheduling with $\mathrm{SI} = 40$ is the best solution to execute the query workload.

2) Performance Evaluation for Different BDAAs. We also evaluate the efficiency of PO for each BDAA with $\mathrm{SI} = 40$. As shown in Fig. 5, for different BDAAs, the resource cost and profit vary significantly. This is caused by variations in the number of admitted queries and their dynamic needs for resources. The results show that PO creates higher profits and generates less resource costs for BDAA1, BDAA3, and BDAA4 compared to HGS. For BDAA2, the profit and resource cost of PO and HGS are the same. We further analyze the results, which show it takes 15.8 hours for PO to execute queries on 7*r3.large VMs while it takes 12.4 hours for HGS to execute queries on 7*r3.large and 1*r3.xlarge VMs. If PO and HGS are used to run a

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

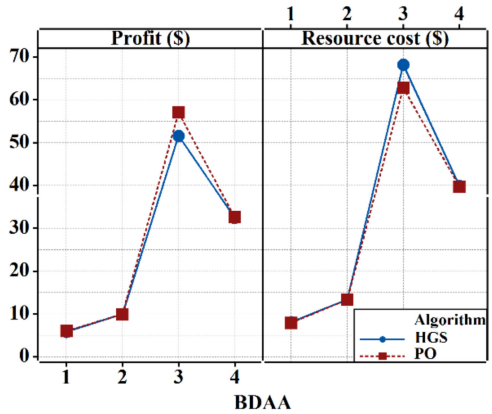

Fig. 5. Profits and resource costs of different BDAAs.

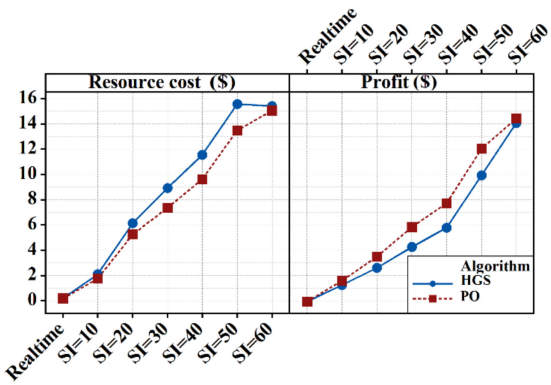

Fig. 6. Profits and resource costs of PO and HGS algorithms for single schedules.

different workload, the performance ad vantages of PO for BDAA2 can be shown.

3) Performance Evaluation for Single Schedules. We further conduct experiments to evaluate advantages of the PO in single schedule without accumulated results from previous schedules. As shown in Fig. 6, for different SIs, the profit created by PO is higher while the resource cost is lower than HGS. We notice a trend in profit enhancement with the increase of SIs as more queries are admitted and PO and HGS gain better knowledge to make scheduling decisions. Experiment results show that PO can generate less resource cost and create higher profit not only for single schedules, but also for each BDAA and the entire query workload for all scheduling scenarios with detailed analysis shown. In summary, we conclude that PO is the best profit optimization algorithm for the AaaS platform compared to HGS, AILP, and AGS for Scheduling Scenario 1.

4.2.3 Resource Configuration

The resource configuration to execute the query workload of PO and HGS is shown in Table 3. The results show that, for all scheduling scenarios, PO can efficiently reduce the numbers of VMs. This is because that PO can make full utilization of the capacity of created resources. To achieve this, PO gives priority to utilize earlier created VMs, which maximizes their utilization and helps to reduce the load in latter created VMs for VM termination during periodic checks. Creating more VMs will cause more delays [23] to serve

TABLE 3
Resource Configuration

Scheduling Scenario	HGS	PO
Real Time	56 * r3. large	36 * r3. large
$SI = 10$	50 * r3. large	33 * r3. large
$SI = 20$	42 * r3. large	30 * r3. large
$SI = 30$	37 * r3. large	25 * r3. large
	1 * r3. xlarge	1 * r3. xlarge
$SI = 40$	32 * r3. large,	
	*1 * r3. xlarge*	26 * r3. large
$SI = 50$	27 * r3. large	22 * r3. large,
	2 * r3. xlarge	2 * r3. xlarge
$SI = 60$	25 * r3. large,	19 * r3. large,
	1 * r3. xlarge	1 * r3. xlarge

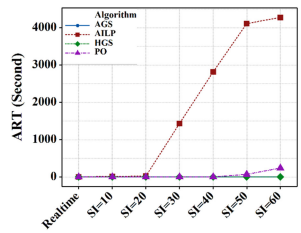

Fig. 7. ART of PO, HGS, AILP, and AGS algorithms.

requests that may cause deadline violations and higher costs. Thus, PO better serves our objectives of SLA guarantee and cost saving.

4.2.4 Algorithm Running Time (ART)

We record the ART for PO, HGS, AILP and AGS to execute the query workload, as shown in Fig. 7. For realtime scheduling, a small number of queries are scheduled based on their arrivals. Thus, ART is not a limiting factor for all algorithms since it only takes a few seconds to schedule queries. For periodic scheduling, it takes few seconds for AGS and HGS to schedule queries for all SIs. For AILP, ART increases rapidly with the increase of SI, which tends to reach a constant value, which is the timeout to run the algorithm. For PO, ART is greatly reduced compared to AILP. For 1 hour scheduling intervals, it takes an average 237 seconds to finish scheduling, which is sufficient fast to solve the scheduling problem. Thus, ART is not a limiting factor for PO to serve as the scheduling algorithm for the AaaS platform to solve the profit optimization problem.

In summary, we conclude that PO is the best profit optimization algorithm of the AaaS platform compared to HGS, AILP, and AGS for Scheduling Scenario 1, which can use less resources, generate less resource cost and create higher profit not only for each schedule, but also for each BDAA and the entire query workload for all realtime and periodic scheduling.

4.3 SLA and Data Splitting Based Resource Scheduling

To overcome the limits of Scheduling Scenario 2 that users have tight deadline and budget constraints of queries, data splitting and sampling are utilized to enable approximate query processing to trade-off accuracy for faster response times and reduced resource costs. It is noted that the SPO

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

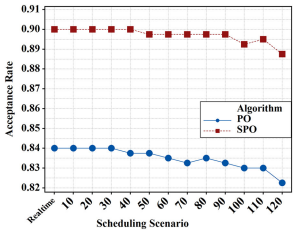

Fig. 8. Admission rates of PO and SPO algorithms.

algorithm serves general purpose by supporting different data splitting and sampling techniques. To illustrate the efficiency and effectiveness of our proposed SPO algorithm, we apply a normal distribution based data splitting method based on our previous work that supports splitting-based big data processing [7]. The ND splitting technique can be replaced by other efficient data processing techniques as long as we can profile the applications to know resource configurations and processing times of the data splitting and sampling methods so that the resource scheduler can make the SLA-based profit optimization decisions. We will continue investigating efficient data splitting and samping techniques to support the AaaS platforms as part of the future work.

The ND-based splitting technique works on normal distributed or near normal distributed datasets that contain at least one numeric column and more numeric columns. It applies a ND based splitting algorithm to generate split datasets as data representatives to improve the accuracy of data analytics. The ND-based method can be efficiently applied to data analysis for medical datasets and enable AaaS platforms to return query results based on a single representative data sample. The representative character of ND-based datasets is evidenced by various medical datasets such as birth weight distribution and body mass index. Applying ND-based splitting to analyze data representatives can provide relatively comprehensive and accurate information than randomly splitting. The inclusive character of ND-based data samples can improve time efficiency, reduce resource costs, and improve accuracy of big data analytics.

The datasets utilized for ND based scheduling are health datasets, which contain the patient information and their hospital records provided by the online sources with records of patients' marriage status, in-hospital duration, etc [7]. The original health datasets are not sufficiently large to apply the data splitting based scheduling method to show its performance benefits. Thus, the required resources for data splitting are enlarged to a suitable scale based on the original datasets. In this study, we generate 1 TB query workload based on big data benchmark with splitting time of health data source as the experimental data. We conduct the following experiments to evaluate the performance of data splitting based resource scheduling and non data-splitting based resource scheduling.

4.3.1 Admission Control and SLA Guarantees

As indicated in Fig. 8, the admission rates of SPO for different scheduling scenarios are in the range of [88, 90 percent], which is approximately 6 percent higher compared to the admission rates of PO in the range of [82, 84 percent]. The fluctuation of the admission rates is caused by the Poisson arrival of the queries. The results show an overall significantly increased query admission rate. The reason is that data-splitting based resource scheduling enables the AaaS platform to admit queries whose analytical solutions can not be generated due to tight deadlines and budgets constraints. Higher query admission rates bring more benefits to the AaaS platform with regards to market share enlargement and profit enhancement.

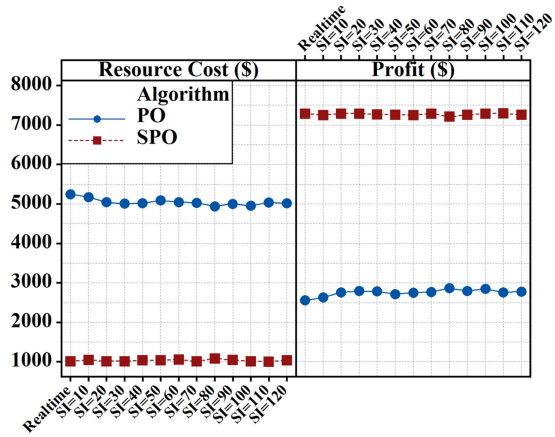

Fig. 9. Resource costs and profits of PO and SPO algorithms.

4.3.2 Profit Enhancement and Cost Saving

As shown in Fig. 9, we analyze the profit enhancement and cost saving, which are the key performance indicators of scheduling algorithms SPO and PO.

1) Average Profits and Resource Costs. With a 6 percent higher acceptance rate, SPO creates an average profit of $7272, which is 2.6 times that of PO with an average resource cost of $1033, which is 20 percent that of PO. The results indicate a significant profit enhancement and cost saving advantage of applying SPO. The reason is that data splitting based scheduling leads to higher query admission rate for higher query incomes as well as a lower resource costs by processing data samples, which subsequently lead to a significant enhancement of profit.

2) Profits and Costs for Different Scenarios. Results show SPO performs significantly better than PO for all scheduling scenarios including realtime scheduling and periodic scheduling with SI from 10 minutes to 2 hours, the profits generated by SPO are in the range of [$7217, $7351] while the profits created by PO are in the range of [$2559, $2863] based on the Amazon EC2 pricing model. The resource costs of PO are [$5246, $4941] while the resource costs of SPO are [$1002, $1088].

3) Performance Evaluation for Single Schedules. We further analyze the performance of PO and SPO for one schedule that is not affected by other scheduling intervals or accumulated results. As shown in Fig. 10, we can see a more rapid increasing trend of profits created by SPO than PO while the resource costs of PO increase much higher than SPO as the scheduling interval increases from realtime to a period of 120 minutes. The results show that for the same scheduling intervals and same query arrival rates, SPO steadily outperforms PO in terms of cost saving and profit enhancement for all scheduling scenarios. We also identify that for realtime scheduling and periodic

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

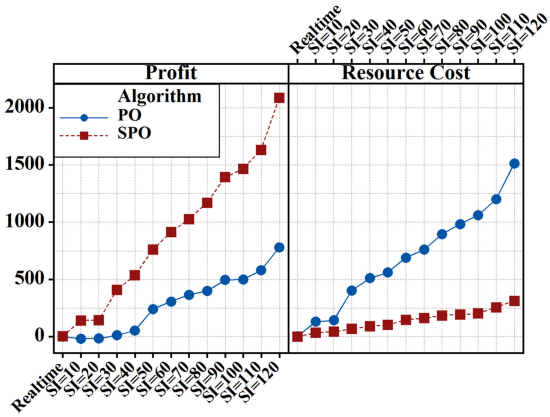

Fig. 10. Profits and resource costs of PO and SPO algorithms for single schedules.

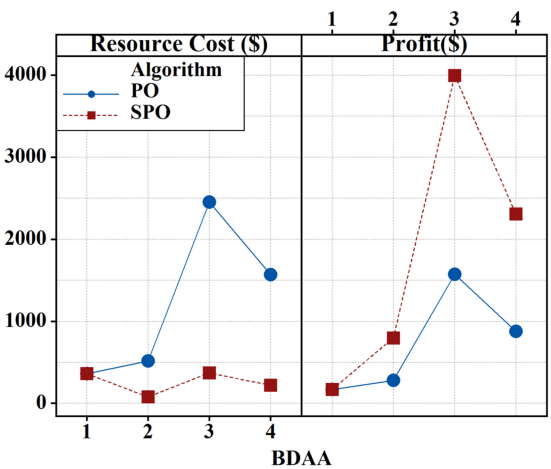

Fig. 11. Profits and resource costs of PO and SPO algorithms for BDAAs with SI = 80.

scheduling with SI of 10 and 20, the profits created by PO includes negative values that indicate that the accepted queries can not fully utilize the cloud resources and the query income can not cover the resource costs. Thus, fully utilizing cloud resources with appropriate SI is important for profit enhancement.

4) Performance Evaluation for Different BDAAs. We use SI = 80 as the scheduling interval to analyze the performance of PO and SPO. It is noted that at SI = 80, SPO generates the lowest profits while PO creates the highest profits. As shown in Fig. 11, as BDAA1 does not enable approximate query processing due to its accuracy requirements, the profit created by SPO is equal to the profit created by PO. For BDAA2, BDAA3, and BDAA4, approximate query processing is allowed to enable quicker responses and less resource costs without strict accuracy requirements. The query acceptance rates of different BDAAs are shown in Table 4. Even for the best performance of PO while at the worst performance of SPO, for BDAA2, BDAA3, and BDAA4, with higher admission rates 10.6, 11.0, and 6.4 percent accordingly, the profits of SPO are 1.8, 1.9, and 1.7 times improved over PO while the resource costs of SPO are 5.3, 6.7, and 6.5 times lower than PO. We conclude that for the other SIs, SPO can better outperform PO for cost saving and profit enhancement.

TABLE 4
Query Admission and SLA Guarantee

Rate	$BDAA_1$	$BDAA_2$	$BDAA_3$	$BDAA_4$
SPO	65.3	98.9	99.0	95.2
PO	65.3	89.5	89.2	89.5

TABLE 5
Resource Configuration

Scheduling Scenario	PO	SPO
Realtime	63 * r3. large	75 * r3. large
SI = 10	59 * r3. large	72 * r3. large
SI = 20	57* r3. large	66 * r3. large
SI = 30	60 * r3. large	63* r3. large
SI = 40	53 * r3. large	58* r3. large
SI = 50	52 * r3. large	59* r3. large
SI = 60	51 * r3. large	55 * r3. large
SI = 70	50* r3. larger 1 * r3.xlarge	53 * r3.larger 1 * r3. xlarge
SI = 80	49* r3. large	53 *r3. large
SI = 90	49* r3.larger 1 * r3. xlarge	52* r3.large, 1 * r3. xlarge
SI = 100	43 * r3.large, 2 * r3. xlarge	46* r3.large, 2 * r3. xlarge
SI = 110 **SI = 120**	41* r3.1large, 1 * r3. xlarge	45 * r3.large, 1 * r3.xlarge

4.3.3 Resource Configuration and Tight Deadline Constraints

We identify that SPO always utilizes a higher number of cloud resources for both realtime scheduling and periodic scheduling, as shown in Table 5. The reason is that SPO admits a higher number of queries than PO. For queries with tight deadlines, new resources are required to execute them simultaneously to avoid deadline violations. PO and SPO both make optimal resource scheduling solutions. For PO, as a profit optimization algorithm, it has already made the optimal scheduling solution to maximize the profits, minimize query processing times, and maximize resource utilization based on relatively less admitted queries compared to SPO. For SPO, although data splitting based sampling enables the processing of smaller data samples to save resource costs whilst reducing the required resource times, the benefits of resource time deduction due to less query processing can not beat the tight deadline constraints of query processing. Despite the fact that creating fewer resources is benefitial and preferable for AaaS platforms, SPO creates a higher number of cloud resources for SLA guarantees to meet tight deadline requirements.

4.3.4 Workload Running Time

Query workload running is the time for the AaaS platform to finish processing the entire query workload. Results indicate that SPO can finish processing the query workload within 500 hours for both realtime and periodic scheduling, as shown in Fig. 12. However, for PO it takes from 1750 hours to 2500 hours to process the queries, which is 3 to 5 times that of SPO. The results show that SPO can significantly reduce query processing times, which better serves the goals of

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

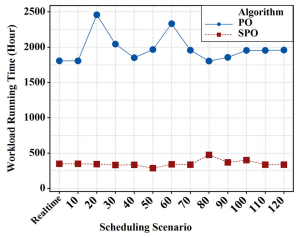

Fig. 12. Workload running times of PO and SPO algorithms

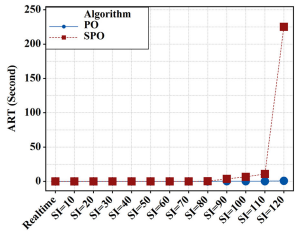

Fig. 13. ART of PO and SPO algorithms.

AaaS platforms for performance enhancements and improve user satisfaction.

4.3.5 Algorithm Running Time

For ART, as shown in Fig. 13, we observe that both PO and SPO can produce scheduling solutions in a timely manner for both realtime and periodic scheduling with SI from 10 minutes and 120 minutes. The average ART for PO for different SIs is between [0.011, 0.767] seconds, while the ART for SPO is between [0.007, 225.313] seconds. We note that ART for PO exhibits a relatively stable trend while we notice a significant increase in the average ART of SPO for $\mathrm{SI} = 72$. Through further analysis of the experiement results, we identify two scheduling intervals reached by the system-defined scheduling timeout. The returned scheduling solutions of SPO may be sub-optimal solutions it has found thusfar to fulfill SLA requirements.

For unexpected extreme high numbers of query arrivals, if there is no optimal solution found, both PO and SPO can switch to heuristic methods to generate scheduling solutions to fulfill tight deadline requirements as the heuristic algorithms that have been refined multiple times can deliver near-optimal scheduling solutions. The ART is not a limiting factor for both SPO and PO to deliver timely and satisfactory scheduling solutions for all use cases.

We focus on reducing the ART of the optimization scheduling algorithms through a refined problem formulation that introduces less variables and constraints in the ILP model with reduced input scales by applying several heuristic algorithms to input near-optimal resource configurations to PO and SPO. The reduced problem and input scale enable PO and SPO to generate timely profit optimization scheduling solutions for large query arrivals.

5 Related Work

Analyzing big data to find potential insights in the data is essential for individuals, organizations, and governments to make better decisions. AaaS platforms aim to deliver on-demand analytics services to various domains of users at lower cost with SLA guarantees in clouds. Our research focuses on providing effective and efficient scheduling algorithms to support the AaaS platform. We compare our work with most related works and detail the comparisons based on various parameters in Table 6.

TABLE 6
Related Works

Parameter	*6*	*7*	*8*	*24*	*26*	*30*	*31*	*27*	*49*	*50*	*51*	*52*	*53*	Our Work
AaaS	Y	N	Y	N	N	N	N	Y	N	N	N	N	N	Y
Cost saving	Y	Y	Y	Y	Y	Y	Y	Y	N	Y	Y	Y	Y	Y
Budget	Y	N	Y	N	Y	Y	Y	N	N	N	N	N	N	Y
Deadline	Y	N	Y	N	Y	Y	Y	N	N	N	N	N	N	Y
Admission control	Y	N	Y	N	N	N	N	N	N	N	Y	N	N	Y
Scalable provision	Y	N	Y	Y	N	N	N	Y	Y	Y	Y	N	N	Y
Automatic provision	Y	N	Y	N	N	N	N	N	N	N	N	Y	N	Y
Data Locality Aware	Y	N	Y	Y	N	N	Y	N	Y	Y	N	N	Y	Y
Data splitting	N	Y	N	N	N	N	N	N	N	N	N	N	N	Y
Diff-dependencies	N	N	N	N	N	N	N	N	N	N	N	N	N	Y
SLA guarantee	Y	N	Y	N	N	Y	N	Y	N	N	Y	Y	N	Y
Profit optimization	N	N	Y	N	N	N	N	N	N	N	N	N	N	Y

Y: Covered; N: Not Covered.

Zhao et al. [6] propose an admission control and resource scheduling algorithm to increase profits for AaaS providers by offering a cost-effective and SLA-guaranteed resource scheduling solution. Zhao et al. [8] further propose a profit-optimization scheduling algorithm to maximize profits for the AaaS platform while minimizing query response times with SLA guarantees. This work futher address the challenge of resource scheduling for big data processing with limited deadlines and budgets by proposing a data-splitting and task-dependency based admission control and profit optimization algorithm.

Mian et al. [24] focus on resource provisioning for data analytic workloads in a public cloud that aims to find cost-effective configuration using heuristic scheduling algorithms. Their work schedules queries for one application while allowing SLA violations to trade-for a cheaper resource cost, which is different from our SLA guaranteeing perspective. Sfrent et al. [25] propose asymptotic scheduling for many tasks computing in big data platforms to maximize the number of the jobs that can be processed and do not consider SLA guarantees. Sun et al. [5] propose a general-purpose analytic framework to provision cost-effective solution for multi-tenancy scenarios. However, they do not consider admission control or address scheduling algorithms with SLA guarantees.

There are extensive works on workflow scheduling. Kllapi et al. [26] address the traditional workflow scheduling problem with time and money optimality. Zulkernine et al. [27] propose a conceptual architecture of cloud-based Analytics-as-a-Service, which does not consider admission control. Zhang et al. [28] propose multi-objective scheduling for many tasks to reduce overhead times and finding a near optimal solution.

Resource provisioning for MapReduce tasks is a well-explored research area, which targets at a specific application model and does not serve general data analytics purposes. Tang et al. [29] propose a self-adaptive scheduling algorithm to reduce the start time of the reduce task of MapReduce. Alrokayan et al. [30] adopt Lambda architecture and focus

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

their work on the batch layer to offer cost-aware and SLA-based algorithms for scheduling MapReduce tasks. Bicer et al. [31] propose a modeling-driven resource allocation framework to support time and cost sensitive execution for MapReduce applications.

There have been serveral cloud enterprises delivering data analytics services. Amazon offers data analytics web services such as Elastic MapReduce [32] that is a service to enable easily and cost-effectively process of big data. Amazon charges users for the cloud resources and the managed MapReduce services. Google offers Big Query [33], which is a fully managed Database-as-a-Service for large-scale data analytics. IBM provides BigInsights [34], which is a Hadoop-as-a-Service to offers the performance and security of an on-premises deployment.

There have been a number of research works focus on leveraging Database-as-a-Service. Relational Cloud [35] is a project that investigates technologies and challenges related to Database-as-a-Service in clouds. Agarwal et al. [36] propose BlinkDB for approximate query processing on large datasets with bounded response time and bounded errors. Narasayya et al. [37] propose SQLVM to provision multi-tenant relational Database-as-a-Service in multi-tenant Microsoft Azure SQL Database. Ahmad et al. [38] propose interaction-aware scheduling for report generation workloads to maximize database performance.

Li et al. [39] propose Big Provision, a provisioning framework, to effectively configure big data analytics systems. Chen et al. [40] address the challenges of delivering continuous Analytics-as-a-Service for realtime events. Wong et al. [41] propose Thrifty for parallel processing Database-as-a-Service to achieve a lower total cost with SLA guarantees on query results.

There are several widely used data analytics frameworks that support big data processing. The big data analytic frameworks that are studied in our work are Shark, Hive, Impala, and Tez/Stinger, which are built on top of the Hadoop Distributed File System [42]. Hadoop is an open-source software framework for reliable, scalable, and distributed computing. Shark is a new module in Spark, which allows interactive SQL on Hadoop systems [43]. Spark is an open-source fast computing framework to provide an easy-to-use programming model and leverages various applications [44]. Hive provides ad hoc query processing and summarizing of data on a data warehouse infrastructure [45]. Impala brings scalable parallel database technology to Hadoop, enabling multi-tenancy and low-latency SQL queries [46]. Tez offers fast processing speed and data scalability to execute batch and interactive tasks [47]. Ousterhout et al. [48] systematically identify the performance bottlenecks of data analytics frameworks to quantify bottlenecks to enable researchers and practitioners make performance improvements.

There have been a number of related works for resource scheduling focus on different perspectives, scenarios, and techniques. Wang et al. [49] proposed a data-aware work stealing technique that is able to achieve efficient load balancing with data-aware scheduling. Xia et al. [50] studied a collaboration- and fairness-aware big data management problem to fairly place continuously generated data to distributed datacenters. Garg et al. [51] manage resources for heterogeneous workload in a datacenter and schedule resources for mixed workloads. Garcia et al. [52] introduce Cloudcompaas, a SLA-aware PaaS cloud platform to manage the complete resource lifecycle and offers a generic SLA model for cloud providers. However, this work does not consider big data analytics but general cloud computing applications. Among the related research works, Gu et al. [53] study the cost minimization problem for big data processing in distributed cloud datacenters. However, this work does not consider QoS requirements, SLA, or cost optimization.

There are an extensive number of works focusing on delivering specific Analytics-as-a-Services, which enriches the BDAAs and services of AaaS platforms. Zhang et al. [7] propose ND-based data splitting methods for large medical data processing in clouds. This approach splits large datasets to obtain smaller representative data for processing thereby offering higher flexibility, less resource costs, and faster response times. Tordini et al. [54], [55] proposed data splitting techniques to process next generation sequencing datasets for complex workflows, which keep a high level of query accuracy.

6 Conclusion and Future Directions

Resource scheduling has significant meaning for AaaS platforms to maximize profits while delivering satisfactory AaaS to users with SLA guarantees. We model and formulate the data-splitting based resource scheduling problem. Moreover, we propose the admission control and profit optimization resource scheduling algorithms, which enable big data analytics with limited budgets and tight deadlines and provide profit optimized scheduling solutions with minimized query times and SLA guarantees. Furthermore, we conduct extensive experiments to evaluate the performance of the proposed algoritms under different scheduling scenarios including realtime and periodic scheduling with varied SIs. Experimental evaluations show the admission control algorithm allows effective query admission while PO performs significantly better than HGS, AILP, and AGS in cost saving and profit enhancement in single schedule, for each BDAA, and for the entire query workload. PO can better serve the profit maximization objectives while delivering AaaS services with SLA guarantees for query processing with full accuracy requirements for Scheduling Scenario 1.

Moreover, we evaluate the performance of SPO with comparison to PO. The results show SPO significantly increases the profits, reduces the resource costs, increases query admission rates, and decreases query processing times. SPO is a promising scheduling algorithm that leverages big data processing with limited budgets and tight deadlines and enables users to obtain time and cost effective analytical solutions for efficient decision making for Scheduling Scenario 2.

As part of the future work, we will continue working on 1) studying approximation query processing, 2) proposing effective data splitting and sampling techniques, 3) proposing SLA-based resource scheduling algorithms to allow query processing on large datasets with faster response times and reduced resource costs.

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

Acknowledgments

The authors wish to thank Dr. Bingsheng He and the reviewers of 2016 IEEE International Conference on Big Data for their helpful feedback.

References

[1] S. Lavalle, E. Lesser, R. Shockley, M. S. Hopkins, and N. Kruschwitz, "Big data, analytics and the path from insights to value," *MIT Sloan Mana. Rev.*, vol. 52, no. 2, pp. 21–32, 2011.
[2] X. Jin, B. W. Wah, X. Cheng, and Y. Wang, "Significance and challenges of big data research," *Big Data Res.*, vol. 2, no. 2, pp. 59–64, 2015.
[3] I. A. T. Hashem, I. Yaqoob, N. Badrul Anuar, S. Mokhtar, A. Gani, and S. Ullah Khan, "The rise of 'big data' on cloud computing: Review and open research issues," *Inf. Syst.*, vol. 47, pp. 98–115, 2014.
[4] C. Ji, Y. Li, W. Qiu, U. Awada, and K. Li, "Big data processing in cloud computing environments," in *Proc. Int. Symp. Pervasive Syst. Algorithms Netw.*, 2012, pp. 17–23.
[5] X. Sun, B. Gao, L. Fan, and W. An, "A cost-effective approach to delivering analytics as a service," in *Proc. IEEE 19th Int. Conf. Web Services*, 2012, pp. 512–519.
[6] Y. Zhao, R. N. Calheiros, G. Gange, K. Ramamohanarao, and R. Buyya, "SLA-Based resource scheduling for big data analytics as a service in cloud computing environments," in *Proc. 44th IEEE Int. Conf. Parallel Process.*, 2015, pp. 510–519.
[7] H. Zhang, Y. Zhao, C. Pang, and J. He, "Splitting large medical data sets based on normal distribution in cloud environment," *IEEE Trans. Cloud Comput.*, 2015, doi: 10.1109/TCC.2015.2462361.
[8] Y. Zhao, R. N. Calheiros, J. Bailey, and R. Sinnott, "SLA-based profit optimization for resource management of big data analytics-as-a-service platforms in cloud computing environments," in *Proc. IEEE Int. Conf. Big Data*, 2016, pp. 432–441.
[9] [Online]. Available: http://spark.apache.org
[10] R. Benayoun, J. De Montgolfier, J. Tergny, and O. Laritchev, "Linear programming with multiple objective functions: Step method (STEM)," *Math. Program.*, vol. 1, pp. 366–375, 1971.
[11] M. Benichou, J. Gauthier, and P. Girodet, "Experiments in mixed-integer linear programming," *Math. Program.*, vol. 1, no. 1, pp. 76–94, 1971.
[12] H. Isermann, "Linear lexicographic optimization," *OR Spektrum*, vol. 4, pp. 223–228, 1982.
[13] [Online]. Available: http://aws.amazon.com/ec2/instance-types/
[14] [Online]. Available: https://amplab.cs.berkeley.edu/benchmark/
[15] M. K. A. B. V. Bittorf, T. Bobrovytsky, C. C. A. C. J. Erickson, M. G. D. Hecht, M. J. I. J. L. Kuff, D. K. A. Leblang, N. L. I. P. H. Robinson, D. R. S. Rus, J. R. D. T. S. Wanderman, and M. M. Yoder, "Impala: A modern, open-source SQL engine for hadoop," in *Proc. 7th Biennial Conf. Innovative Data Syst. Res.*, 2015.
[16] S. Shenker, I. Stoica, M. Zaharia, and R. Xin, "Shark: SQL and rich analytics at scale," in *Proc. ACM SIGMOD Int. Conf. Mana. Data*, 2013, pp. 13–24.
[17] A. Thusoo, J. Sarma, N. Jain, and Z. Shao, "Hive: A warehousing solution over a map-reduce framework," *Proc. VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.
[18] B. Saha, H. Shah, S. Seth, G. Vijayaraghavan, A. Murthy, and C. Curino, "Apache tez: A unifying framework for modeling and building data processing applications," in *Proc. ACM SIGMOD Int. Conf. Mana. Data*, 2015, pp. 1357–1369.
[19] J. Schad, J. Dittrich, and J.-A. Quiané-Ruiz, "Runtime measurements in the cloud: Observing, analyzing, and reducing variance," *Proc. VLDB Endow*, vol. 3, no. 1–2, pp. 460–471, 2010.
[20] R. N. Calheiros and R. Buyya, "Cost-effective provisioning and scheduling of deadline-constrained applications in hybrid clouds," in *Proc. Int. Conf. Web Inf. Syst. Engineering*, 2012, pp. 171–184.
[21] [Online]. Available: https://www.ibm.com/developerworks/downloads/ws/ilogcplex/
[22] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Softw.: Practice Experience*, vol. 41, no. 1, pp. 23–50, 2011.
[23] M. Mao and M. Humphrey, "A performance study on the VM startup time in the cloud," in *Proc. IEEE Fifth Int. Conf. Cloud Comput.*, 2012, pp. 423–430.
[24] R. Mian, P. Martin, and J. Vazquez-Poletti, "Provisioning data analytic workloads in a cloud," *Future Generation Comput. Syst.*, vol. 29, no. 6, pp. 1452–1458, 2013.
[25] A. Sfrent and F. Pop, "Asymptotic scheduling for many task computing in big data platforms," *Inf. Sci.*, vol. 319, pp. 71–91, 2015.
[26] H. Kllapi, E. Sitaridi, M. M. Tsangaris, and Y. Ioannidis, "Schedule optimization for data processing flows on the cloud," in *Proc. Int. Conf. Mana. Data - SIGMOD*, 2011, pp. 289–300.
[27] F. Zulkernine, P. Martin, Y. Zou, M. Bauer, F. Gwadry-Sridhar, and A. Aboulnaga, "Towards cloud-based analytics-as-a-service (CLAaaS) for big data analytics in the cloud," in *Proc. IEEE Int. Congr. Big Data*, 2013, pp. 62–69.
[28] F. Zhang, J. Cao, K. Li, S. U. Khan, and K. Hwang, "Multi-objective scheduling of many tasks in cloud platforms," *Future Generation Comput. Syst.*, vol. 37, pp. 309–320, 2014.
[29] Z. Tang, L. Jiang, J. Zhou, K. Li, and K. Li, "A self-adaptive scheduling algorithm for reduce start time," *Future Generation Comput. Syst.*, vol. 43–44, pp. 51–60, 2015.
[30] M. Alrokayan, A. Vahid Dastjerdi and R. Buyya, "SLA-aware provisioning and scheduling of cloud resources for big data analytics," in *Prof. IEEE Int. Conf. Cloud Comput. Emerging Markets*, 2014, pp. 1–8.
[31] T. Bicer, D. Chiu, and G. Agrawal, "Time and cost sensitive data-intensive computing on hybrid clouds," in *Proc. 12th IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, 2012, pp. 636–643.
[32] [Online]. Available: https://aws.amazon.com/elasticmapreduce/
[33] [Online]. Available: https://Cloud.google.com/bigquery/
[34] [Online]. Available: http://www-03.ibm.com/software/products/en/ibm-biginsights-for-apache-hadoop
[35] [Online]. Available: http://relationalcloud.com
[36] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica, "BlinkDB: Queries with bounded errors and bounded response times on very large data," in *Proc. 8th ACM Eur. Conf. Comput. Syst.*, 2013, Art. no. 29.
[37] V. Narasayya, S. Das, M. Syamala, B. Chandramouli, and S. Chaudhuri, "SQLVM: Performance isolation in multi-tenant relational database-as-a-service," in *Proc. 6th Biennial Conf. Innovative Data Syst. Res.*, 2013.
[38] M. Ahmad, A. Aboulnaga, S. Babu, and K. Munagala, "Interaction-aware scheduling of report-generation workloads," *VLDB J.*, vol. 20, no. 4, pp. 589–615, 2011.
[39] H. Li, K. Lu, and S. Meng, "BigProvision: A provisioning framework for big data analytics," *IEEE Netw.*, vol. 29, no. 5, pp. 50–56, Sep.-Oct. 2015.
[40] Q. Chen, M. Hsu, and H. Zeller, "Experience in continuous analytics as a service," in *Proc. 14th Int. Conf. Extending Database Technol.*, 2011, Art. no. 509.
[41] P. Wong, Z. He, and E. Lo, "Parallel analytics as a service," in *Proc. Int. Conf. Mana. Data - SIGMOD*, 2013, Art. no. 25.
[42] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Syst. Technol.*, 2010, pp. 1–10.
[43] C. Engle, A. Lupher, R. Xin, M. Zaharia, M. J. Franklin, S. Shenker, and I. Stoica, "Shark," in *Proc. Int. Conf. Mana. Data*, 2012, Art. no. 689.
[44] M. Zaharia, M. Chowdhury, T. Das, A. Dave, and J. Ma, "Fast and interactive analytics over hadoop data with spark," *USENIX Login*, vol. 37, no. 4: pp. 45–51, 2012.
[45] A. Thusoo, J. Sen Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, and R. Murthy, "Hive," *Proc. VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, Aug. 2009.
[46] T. Liu, M. Martonosi, T. Liu, and M. Martonosi, "Impala: A middleware system for managing autonomic, parallel sensor systems ting," in *Proc. 9th ACM SIGPLAN Symp. Principles Practice Parallel Program.*, 2003, Art. no. 107.
[47] [Online]. Available: http://zh.hortonworks.com/hadoop/tez/
[48] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, "Sparrow : Distributed, low latency scheduling," in *Proc. ACM Symp. Operating Syst. Principles*, 2013, pp. 69–84k.
[49] K. Wang, X. Zhou, T. Li, D. Zhao, M. Lang, and I. Raicu, "Optimizing load balancing and data-locality with data-aware scheduling," in *Proc. IEEE Int. Conf. Big Data*, 2015, pp. 119–128.
[50] Q. Xia, Z. Xu, W. Liang, and A. Y. Zomaya, "Collaboration- and fairness-aware big data management in distributed clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 7, pp. 1941–1953, Jul. 2015.
[51] S. K. Garg, A. N. Toosi, S. K. Gopalaiyengar, and R. Buyya, "SLA-based virtual machine management for heterogeneous workloads in a cloud datacenter," *J. Netw. Comput. Appl.*, vol. 45, pp. 108–120, 2014.

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.

[52] A. G. Garcia, I. Blanquer Espert, and V. Hernández García, "SLA-driven dynamic cloud resource management," *Future Generation Comput. Syst.*, vol. 31, pp. 1–11, 2014.
[53] L. Gu, D. Zeng, P. Li, and S. Guo, "Cost minimization for big data processing in geo-distributed data centers," *IEEE Trans. Emerging Topics Comput.*, vol. 2, no. 3, pp. 314–323, Sep. 2014.
[54] F. Tordini, M. Aldinucci, P. Viviani, I. Merelli, and P. Lio, "Scientific workflows on clouds with heterogeneous and preemptible instances," in *Proc. Int. Conf. Parallel Comput.*, 2017, pp. 605–614.
[55] F. Tordini, "A cloud solution for multiomics data integration," in *Proc. 16th IEEE Int. Conf. Scalable Comput. Commun.*, 2016, pp. 559–566.

Yali Zhao received the bachelor's degree in software engineering, the master's degree in software engineering, the master's degree in enterprise computing and engineering, she is currently working toward the PhD degree in the University of Melbourne, Australia. Her research interest is SLA-based resource scheduling for big data analytic-as-a-service in cloud computing environments.

Rodrigo N. Calheiros is a lecturer in the School of Computing, Engineering and Mathematics, Western Sydney University, Australia. He works in the field of Cloud computing and related areas since 2008, and since then he carried out R&D supporting research in the area. His research interests also include big data, internet of things, and their application.

Graeme Gange is a lecturer at Monash University. His research interests include discrete optimization, particularly integration of constraint programming, SAT and SMT techniques, as well as development of static analysis and program verification techniques.

James Bailey received the PhD degree from the University of Melbourne, in 1998. He is currently an Australian Research Council Future fellow in the Department of Computing and Information Systems, University of Melbourne. He has previously held appointments at Kings College, Birkbeck College, and The University of London. His research interests are in the area of data mining and machine learning, with a focus on both fundamental topics such as contrast pattern mining and data clustering, as well as application aspects in areas such as health informatics and bioinformatics.

Richard O. Sinnott received the BSc degree in theoretical physics (Hons), the MSc degree in software engineering and the PhD degree in computing science. He is the director of eResearch at the University of Melbourne and professor of Applied Computing Systems. He has been lead software engineer/architect on an extensive portfolio of national and international projects, with specific focus on research domains requiring finer-grained access control (security). Prior to coming to Melbourne, he was the Technical director of the UK National e-Science Centre; director of e-Science at the University of Glasgow; deputy director (Technical) for the Bioinformatics Research Centre also at the University of Glasgow, and for a while the technical director of the National Centre for e-Social Science. He has more than 300 peer-reviewed publications across a range of computing and application-specific domains.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 14:58:21 UTC from IEEE Xplore. Restrictions apply.